# Overview of Reinforcement learning

*Subject: Reinforcement learning*

---

**1.**

Deep reinforcement learning This approach extends reinforcement learning by using a deep neural network and without explicitly designing the state space. The work on learning ATARI games by Google DeepMind increased attention to deep reinforcement learning or end-to-end reinforcement learning.

**2.**

Sample inefficiency RL algorithms often require a large number of interactions with the environment to learn effective policies, leading to high computational costs and time-intensive to train the agent. For instance, OpenAI's Dota-playing bot utilized thousands of years of simulated gameplay to achieve human-level performance. Techniques like experience replay and curriculum learning have been proposed to deprive sample inefficiency, but these techniques add more complexity and are not always sufficient for real-world applications.

**3.**

A policy that achieves these optimal state-values in each state is called optimal. Clearly, a policy that is optimal in this sense is also optimal in the sense that it maximizes the expected discounted return, since V ∗ ( s ) = max π E [ G ■ s , π ] {\displaystyle V^{*}(s)=\max _{\pi }\mathbb {E} [G\mid s,\pi ]} , where s {\displaystyle s} is a state randomly sampled from the distribution μ {\displaystyle \mu } of initial states (so μ ( s ) = Pr ( S 0 = s ) {\displaystyle \mu (s)=\Pr(S_{0}=s)} ). Although state-values suffice to define optimality, it is useful to define action-values. Given a state s {\displaystyle s} , an action a {\displaystyle a} and a policy π {\displaystyle \pi } , the action-value of the pair ( s , a ) {\displaystyle (s,a)} under π {\displaystyle \pi } is defined by

**4.**

The environment is typically stated in the form of a Markov decision process, as many reinforcement learning algorithms use dynamic programming techniques. The main difference between classical dynamic programming methods and reinforcement learning algorithms is that the latter do not assume knowledge of an exact mathematical model of the Markov decision process, and they target large Markov decision processes where exact methods become infeasible.

**5.**

Associative reinforcement learning Associative reinforcement learning tasks combine facets of stochastic learning automata tasks and supervised learning pattern classification tasks. In associative reinforcement learning tasks, the learning system interacts in a closed loop with its environment.

**6.**

where G {\displaystyle G} stands for the discounted return associated with following π {\displaystyle \pi } from the initial state s {\displaystyle s} . Defining V ∗ ( s ) {\displaystyle V^{*}(s)} as the maximum possible state-value of V π ( s ) {\displaystyle V^{\pi }(s)} , where π {\displaystyle \pi } is allowed to change,

**7.**

An alternative method is to search directly in (some subset of) the policy space, in which case the problem becomes a case of stochastic optimization. The two approaches available are gradient-based and gradient-free methods. Gradient-based methods (policy gradient methods) start with a mapping from a finite-dimensional (parameter) space to the space of policies: given the parameter vector θ {\displaystyle \theta } , let π θ {\displaystyle \pi _{\theta }} denote the policy associated to θ {\displaystyle \theta } . Defining the performance function by ρ ( θ ) = ρ π θ {\displaystyle \rho (\theta )=\rho ^{\pi _{\theta }}} under mild conditions this function will be differentiable as a function of the parameter vector θ {\displaystyle \theta } . If the gradient of ρ {\displaystyle \rho } was known, one could use gradient ascent. Since an analytic expression for the gradient is not available, only a noisy estimate is available. Such an estimate can be constructed in many ways, giving rise to algorithms such as Williams's REINFORCE method (which is known as the likelihood ratio method in the simulation-based optimization literature). A large class of methods avoids relying on gradient information. These include simulated annealing, cross-entropy search or methods of evolutionary computation. Many gradient-free methods can achieve (in theory and in the limit) a global optimum. Policy search methods may converge slowly given noisy data. For example, this happens in episodic problems when the trajectories are long and the variance of the returns is large. Value-function based methods that rely on temporal differences might help in this case. In recent years, actor–critic methods have been proposed and performed well on various problems. Policy search methods have been used in the robotics context. Many policy search methods may get stuck in local optima (as they are based on local search).

**8.**

Algorithms for control learning Even if the issue of exploration is disregarded and even if the state was observable (assumed hereafter), the problem remains to use past experience to find out which actions lead to higher cumulative rewards.

**9.**

where R t + 1 {\displaystyle R_{t+1}} is the reward for transitioning from state S t {\displaystyle S_{t}} to S t + 1 {\displaystyle S_{t+1}} , 0 ≤ γ < 1 {\displaystyle 0\leq \gamma <1} is the discount rate. γ {\displaystyle \gamma } is less than 1, so rewards in the distant future are weighted less than rewards in the immediate future. The algorithm must find a policy with maximum expected discounted return. From the theory of Markov decision processes it is known that, without loss of generality, the search can be restricted to the set of so-called stationary policies. A policy is stationary if the action-distribution returned by it depends only on the last state visited (from the observation agent's history). The search can be further restricted to deterministic stationary policies. A deterministic stationary policy deterministically selects actions based on the current state. Since any such policy can be identified with a mapping from the set of states to the set of actions, these policies can be identified with such mappings with no loss of generality.

**10.**

Function approximation methods In order to address the fifth issue, function approximation methods are used. Linear function approximation starts with a mapping φ {\displaystyle \phi } that assigns a finite-dimensional vector to each state-action pair. Then, the action values of a state-action pair ( s , a ) {\displaystyle (s,a)} are obtained by linearly combining the components of φ ( s , a ) {\displaystyle \phi (s,a)} with some weights θ {\displaystyle \theta } :